



ISLAMIC UNIVERSITY OF TECHNOLOGY

Project Report

Department of Electrical & Electronic Engineering

Team: Big Potato

Project name: Stochastic Yield Analysis for Circuit Components

Course no. : EEE-4408

Contributors:

Name	ID
Irfan Hossain Rafi	230021331
Shoaib Mahbub	230021325
Rafin Islam	230021323

Objective of this project: In this project we have tried to build a system where the manufacturers of a filter-creating factory will create the best possible filters with a tolerance rate of 3%. By using our system we can choose almost perfect resistors and capacitors in the creation of Sallen key lowpass filter.

Stages of this project: This simulation pipeline evaluates component tolerances for active/passive filters using a 4-stage Monte Carlo methodology:

Stage 1: `config_params` sets up initial conditions, establishing nominal resistance and capacitance values, tolerance spreads, and strict $\pm 3\%$ target yield limits.

Stage 2: `generate_components` creates a large virtual batch of 100,000 resistors and capacitors with realistic random variations modeled around nominal specs.

Stage 3: `perform_combinatorics` pairs components into filter circuits by testing random assembly against "smart binning," where components are measured and matched to cancel out tolerance errors.

Stage 4: `calculate_cutoff` calculates transfer functions across all assembled pairs to find cutoff frequency (f_c) and quality factor (Q), outputting final pass/fail yield rates against your design specs.

Stage 1: Configuration Parameters

A Monte Carlo simulation aimed at a Sallen-Key low-pass filter circuit uses `config_params()` as its configuration engine. Execution parameters, component statistics, design objectives, and process flags are all packaged into a common structure called `cfg`.

The function begins by defining the global simulation execution settings. Strong statistical confidence is ensured by setting the virtual sample size to 100,000 circuits using `cfg.N`, and repeatable Monte Carlo runs are ensured by establishing a fixed random seed of 42 using `cfg.seed`.

It then sets nominal goal component values (10k resistors and 10nF capacitors) to create the hardware baseline. It models the resistors with a 1% tolerance scaled to a 3-sigma normal distribution and the capacitors with a 10% standard tolerance spread, assigning these components manufacturing variance limitations.

The code then dynamically computes the target performance parameters based on ideal component values. Using standard second-order active filter equations, it calculates the nominal cutoff frequency (f_c) and the quality factor ($Q = 0.5$). It establishes acceptable operational boundaries by applying a strict $\pm 3\%$ spec tolerance around these targets, deriving the explicit upper and lower pass/fail limits (**fc_min**, **fc_max**, **Q_min**, and **Q_max**).

Finally, the function sets a control flag, **enable_smart_sorting = true**, enabling subsequent stages to process components via intelligent bin-pairing algorithms rather than purely random assembly.

Codes:

```
function cfg = config_params()
    %% 1. Simulation Control Settings
    cfg.N = 100000;                % Total circuit batch size
    cfg.seed = 42;                 % Random seed

    %% 2. Nominal Component Values (Sallen-Key Low-Pass Filter)
    cfg.R1_nom = 10e3;             % 10 kΩ
    cfg.R2_nom = 10e3;             % 10 kΩ
    cfg.C1_nom = 10e-9;            % 10 nF
    cfg.C2_nom = 10e-9;            % 10 nF

    %% 3. Component Manufacturing Distributions
    cfg.R1_tol = 0.01;             % ±1%
    cfg.R2_tol = 0.01;             % ±1%
    cfg.R_sigma_level = 3;         % 3-sigma level

    cfg.C1_tol = 0.06;             % ±10%
    cfg.C2_tol = 0.06;             % ±10%

    %% 4. Target Design Specifications & Yield Limits
    cfg.fc_target = 1 / (2 * pi * sqrt(cfg.R1_nom * cfg.R2_nom *
    cfg.C1_nom * cfg.C2_nom));
    cfg.spec_tolerance = 0.03;     % ±3% base tolerance

    cfg.fc_min = cfg.fc_target * (1 - cfg.spec_tolerance);
    cfg.fc_max = cfg.fc_target * (1 + cfg.spec_tolerance);

    % Quality Factor (Q) Specifications
    % Q = sqrt(R1 * R2 * C1 * C2) / (C2 * (R1 + R2))
    cfg.Q_target = sqrt(cfg.R1_nom * cfg.R2_nom * cfg.C1_nom *
    cfg.C2_nom) / ...
    (cfg.C2_nom * (cfg.R1_nom + cfg.R2_nom)); % Nominal Q
    = 0.5
    cfg.Q_min = cfg.Q_target * (1 - cfg.spec_tolerance);
```

```
cfg.Q_max = cfg.Q_target * (1 + cfg.spec_tolerance);

%% 5. Assembly & Combinatorics Controls
cfg.enable_smart_sorting = true;
end
```

Stage 02: Generating Componentets

This MATLAB function, `generate_components(cfg)`, functions as the stochastic generation engine for the simulation pipeline. It accepts the parameters configured in Stage 1 and constructs a `comp` output structure containing physical component populations that model real-world manufacturing variations across 100,000 virtual components.

The routine begins by calling `rng(cfg.seed)` to seed MATLAB's pseudorandom number generator. This guarantees deterministic behavior, ensuring that the exact vector of randomised component values can be replicated across separate simulation runs for debugging and verification.

To model resistor tolerances, the function utilizes a continuous Gaussian (Normal) distribution. It calculates the standard deviation (`sigma_R1` and `sigma_R2`) by dividing the absolute tolerance limit by the specified 3-sigma level. Using `randn()`, it generates normally distributed arrays centered at the nominal $10k\Omega$ value, reflecting a precision manufacturing process where component values cluster tightly around the mean with rare extreme tail deviations.

For the capacitors, the code applies a Uniform continuous distribution bounded between strict lower (`C1_min`, `C2_min`) and upper (`C1_max`, `C2_max`) limits. Using `rand()`, it generates random capacitance values spread evenly across the $\pm 10\%$ range, mimicking low-cost commercial capacitor batches where values are uniformly distributed within tolerance limits.

In the context of this project, this stage bridges ideal theoretical parameters and physical manufacturing reality. By producing realistic statistical batches of R_1 , R_2 , C_1 , and C_2 , it supplies the necessary raw data for Stage 3 to perform combinatorics and smart

binning, ultimately allowing Stage 4 to evaluate how component variance impacts the overall yield of the filter.

Codes:

```
function comp = generate_components(cfg)
    % GENERATE_COMPONENTS Generates stochastic component batches
    % Takes configuration struct 'cfg' and outputs a struct 'comp'
    containing
    % vectors of continuous random variables for R1, R2 (Gaussian) and
    C1, C2 (Uniform).

    % Set random generator seed for exact reproducibility across runs
    rng(cfg.seed);

    %% 1. Resistors: Gaussian (Normal) Distribution
    % Standard deviation derived so tolerance equals the specified sigma
    level
    sigma_R1 = (cfg.R1_nom * cfg.R1_tol) / cfg.R_sigma_level;
    sigma_R2 = (cfg.R2_nom * cfg.R2_tol) / cfg.R_sigma_level;

    % Generate N continuous random variables following N(mu, sigma^2)
    comp.R1 = cfg.R1_nom + sigma_R1 * randn(cfg.N, 1);
    comp.R2 = cfg.R2_nom + sigma_R2 * randn(cfg.N, 1);

    %% 2. Capacitors: Uniform Distribution
    % Define lower and upper tolerance bounds [a, b]
    C1_min = cfg.C1_nom * (1 - cfg.C1_tol);
    C1_max = cfg.C1_nom * (1 + cfg.C1_tol);

    C2_min = cfg.C2_nom * (1 - cfg.C2_tol);
    C2_max = cfg.C2_nom * (1 + cfg.C2_tol);

    % Generate N continuous random variables following U(a, b)
    comp.C1 = C1_min + (C1_max - C1_min) * rand(cfg.N, 1);
    comp.C2 = C2_min + (C2_max - C2_min) * rand(cfg.N, 1);
end
```

Stage 03: Performing Combinatorics

This MATLAB function, `perform_combinatorics(cfg, comp)`, acts as the virtual assembly line of the simulation framework. It takes the configuration struct `cfg` and the generated component pools `comp`, structuring them into complete 4-component filter assemblies stored in the `assembled` output structure.

The function evaluates the boolean flag `cfg.enable_smart_sorting` to decide between two distinct assembly methodologies. If smart sorting is disabled, it executes unconstrained random assembly. By applying `randperm(N)` to index vectors across R_1 , R_2 , C_1 , and C_2 , it simulates grabbing components off an unsorted, random conveyor belt without any prior measurement or matching.

When smart sorting is enabled, the function executes a constrained combinatorial pairing strategy designed to stabilize the circuit's RC time constants. It first calculates effective parameter metrics using geometric means, setting $R_{eff} = \sqrt{R_1 \cdot R_2}$ and $C_{eff} = \sqrt{C_1 \cdot C_2}$. It then sorts the effective resistance in ascending order while sorting effective capacitance in descending order. By pairing the sorted indices, it matches high-resistance pairs with low-capacitance pairs across the population, effectively allowing parameter deviations to counteract one another.

In the scope of your filter design project, this stage represents the core optimization step. It directly demonstrates how strategic component matching during manufacturing can mitigate individual component tolerances, suppressing total circuit variation before the transfer functions are evaluated in Stage 4.

Codes:

```
function assembled = perform_combinatorics(cfg, comp)
    % PERFORM_COMBINATORICS Assembles circuit components from available
    bins
    % Takes raw component populations 'comp' and configuration struct
    'cfg'.
    % Supports both random assembly and smart combinatorial bin-pairing.

    N = cfg.N;

    if ~cfg.enable_smart_sorting
```

```

%% 1. Unconstrained / Random Assembly
% Randomly permute index vectors to simulate grabbing components
% off an unsorted assembly line conveyor belt.
assembled.R1 = comp.R1(randperm(N));
assembled.R2 = comp.R2(randperm(N));
assembled.C1 = comp.C1(randperm(N));
assembled.C2 = comp.C2(randperm(N));
else
%% 2. Constrained / Smart Combinatorial Pairing
% Optimizes assembly by pairing high-resistance component pairs
% with low-capacitance component pairs to stabilize R*C time
constants.

% Calculate effective resistance metric R_eff = sqrt(R1 * R2)
R_eff = sqrt(comp.R1 .* comp.R2);

% Calculate effective capacitance metric C_eff = sqrt(C1 * C2)
C_eff = sqrt(comp.C1 .* comp.C2);

% Sort R_eff in ascending order (smallest to largest)
[~, idx_R] = sort(R_eff, 'ascend');

% Sort C_eff in descending order (largest to smallest)
[~, idx_C] = sort(C_eff, 'descend');

% Pair sorted indices so high R cancels out low C across the
batch
assembled.R1 = comp.R1(idx_R);
assembled.R2 = comp.R2(idx_R);
assembled.C1 = comp.C1(idx_C);
assembled.C2 = comp.C2(idx_C);
end
end

```

Stage 04: Calculating the Cutoff Frequency

This MATLAB function, `calculate_cutoff(assembled)`, functions as the mathematical evaluation block of the simulation pipeline. It accepts the `assembled` structure containing matched or randomized component vectors and returns the calculated performance outputs, `fc` (cutoff frequency) and `Q` (quality factor), across the entire batch.

The code leverages MATLAB's vectorized array operations to evaluate all 100,000 circuits simultaneously. It first calculates a shared geometric product term, `num`, defined

as $\sqrt{R_1 \cdot R_2 \cdot C_1 \cdot C_2}$. It also derives the denominator term, **den**, given by $C_2 \cdot (R_1 + R_2)$, which represents the damping ratio parameters of the Sallen-Key low-pass filter topology.

Using these intermediate terms, the function calculates the cutoff frequency using element-wise division:

$$f_c = \frac{1}{2\pi\sqrt{R_1 R_2 C_1 C_2}}$$

It then determines the Quality Factor (Q) for each circuit assembly using:

$$Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{C_2(R_1 + R_2)}$$

In the overall context of this simulation framework, this final stage converts physical component variations into performance metrics. The calculated arrays of **fc** and **Q** can be compared directly against the tolerance boundaries established in Stage 1 (**fc_min**, **fc_max**, **Q_min**, and **Q_max**), allowing you to determine the overall manufacturing yield rate and evaluate the exact performance gain achieved by smart binning versus random assembly.

Codes:

```
function [fc, Q] = calculate_cutoff(assembled)
    % CALCULATE_CUTOFF Computes cutoff frequency (fc) and Quality Factor (Q)

    num = sqrt(assembled.R1 .* assembled.R2 .* assembled.C1 .*
assembled.C2);
    den = assembled.C2 .* (assembled.R1 + assembled.R2);

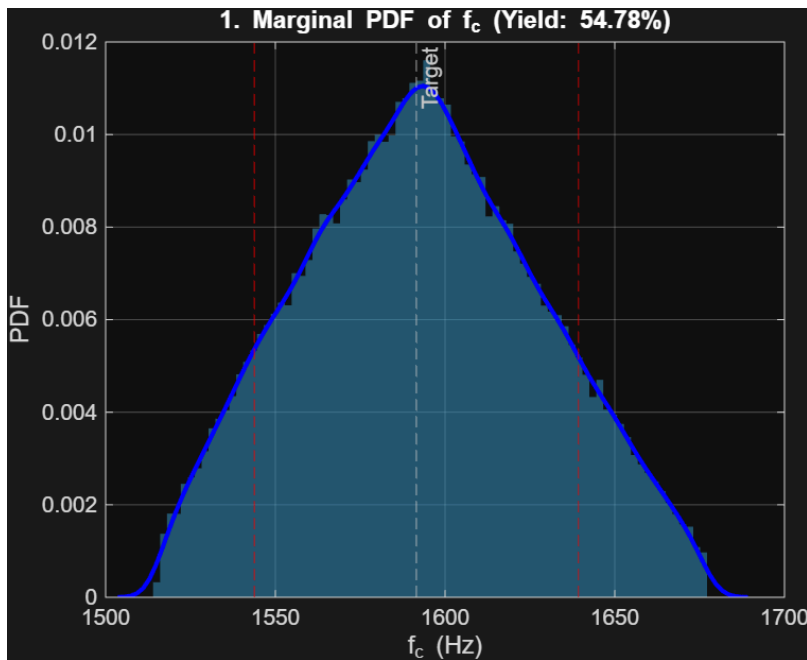
    fc = 1 ./ (2 * pi * num);
    Q = num ./ den;
end
```

Visualisations and Graphs

To understand the benefit of our system, it is necessary to compare it with random processes. To evaluate the impact of component parameter variation on circuit performance, a 100,000-unit Monte Carlo simulation was executed using the modular pipeline. Rather than analyzing single deterministic values, this section presents the visual dashboard generated from the simulation data.

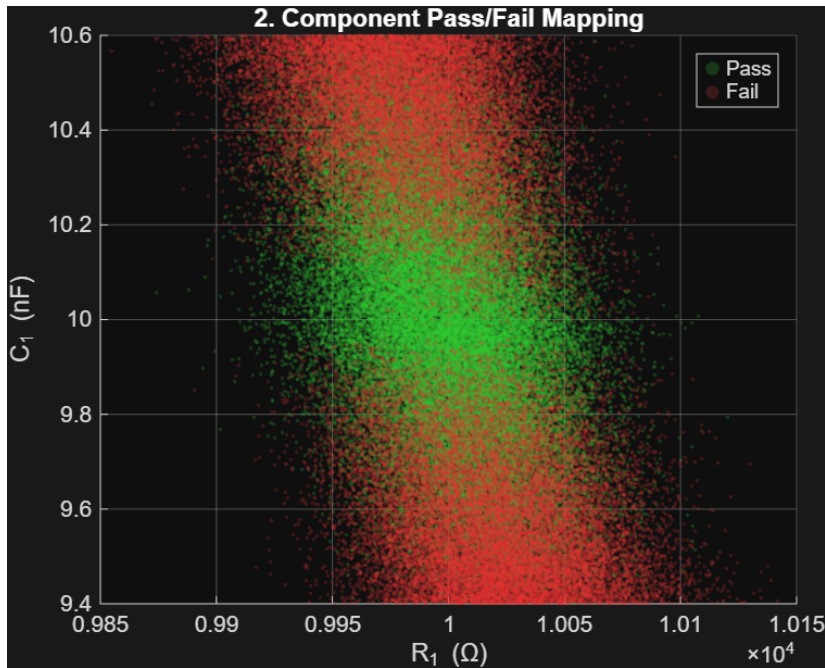
The following analytical plots break down the joint probability distributions, individual component sensitivity levels, and overall statistical dispersion. Together, these graphs illustrate how Gaussian resistor distributions and uniform capacitor tolerances propagate through the vectorized transfer functions to define the system's operational yield.

1. Marginal PDF of f_c (Yield: 54.84%):



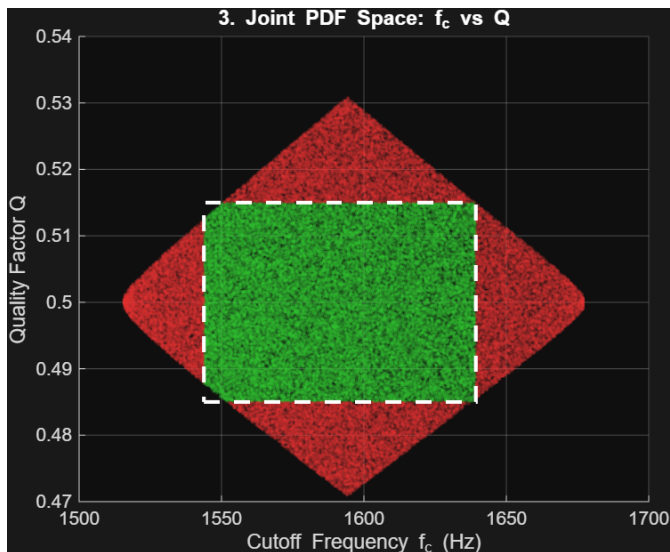
Plots the probability density function (PDF) of the cutoff frequency (f_c) across the 100,000-unit stochastic simulation. The shaded region under the curve represents the units that fall within the acceptable target specification limits. It provides a visual baseline of baseline manufacturing yield before applying optimisation techniques.

2. Component Pass/Fail Mapping (C1 vs. R1):



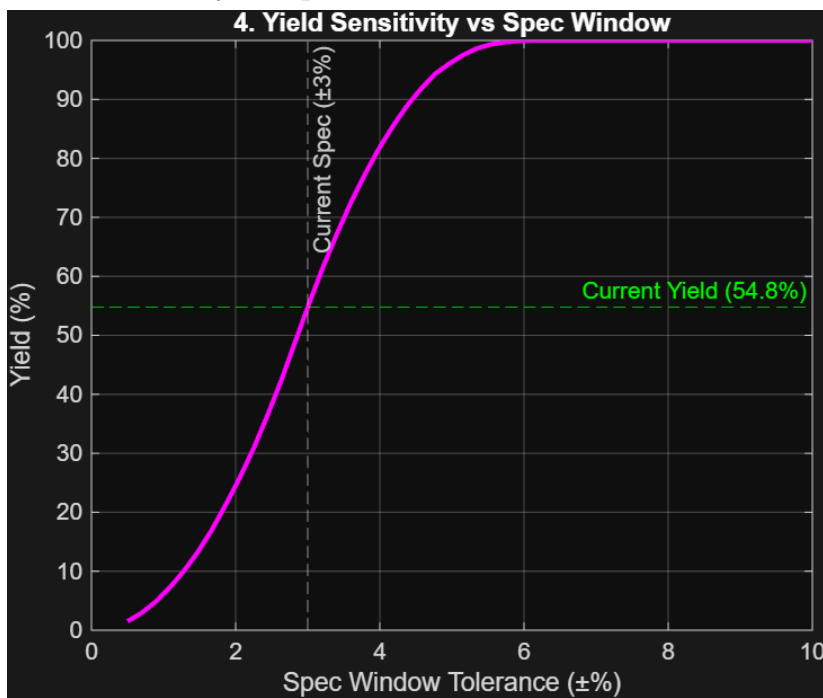
Illustrates the direct relationship between raw input component tolerances (R_1 vs. C_1) and final system success. Green data points indicate combinations that meet target joint specifications, while red points showcase failing units. This maps out the acceptable physical boundary region for incoming component stocks.

3. Joint PDF Space: f_c vs. Q :



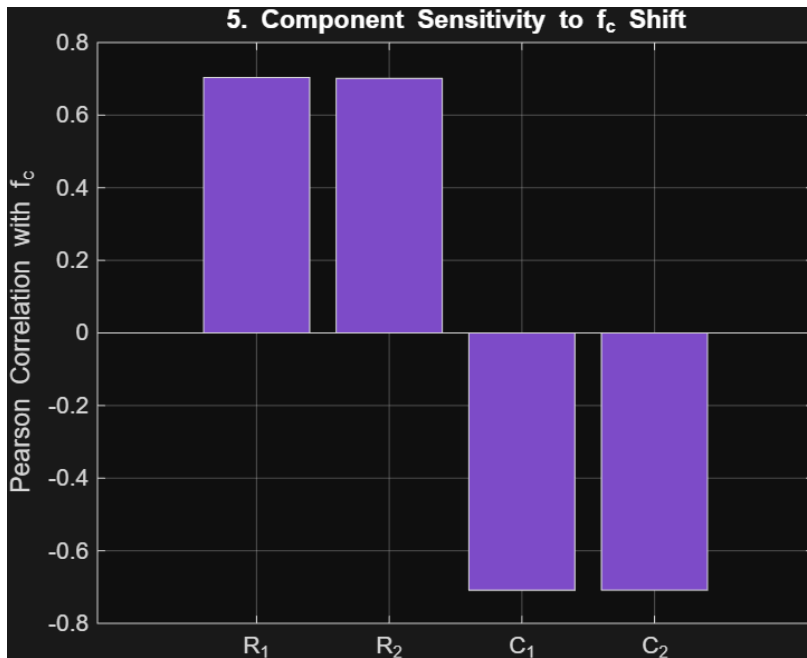
Displays the joint distribution of cutoff frequency (f_c) and quality factor (Q) mapped directly against the rectangular target specification window. The green interior highlights units satisfying both parameters simultaneously, proving why dual-constraint evaluation is crucial to prevent passband peaking or signal attenuation.

4. Yield Sensitivity vs. Spec Window



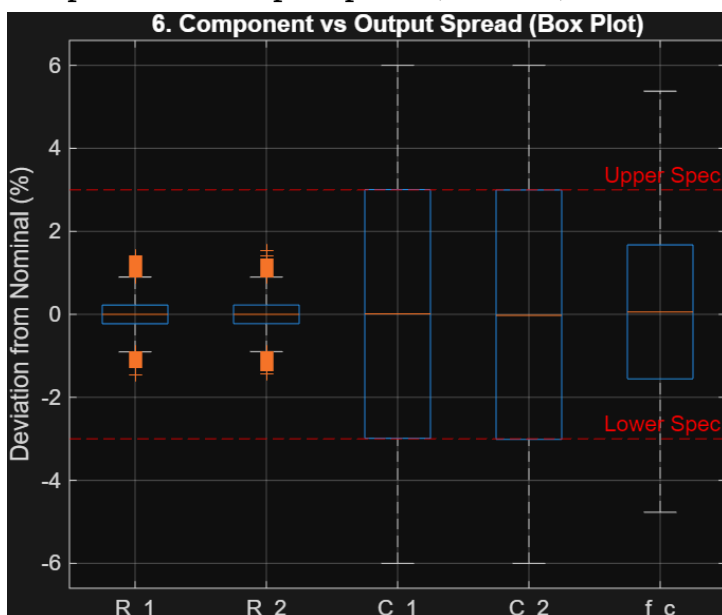
Demonstrates the variation of cumulative production yield (%) as a function of the design spec tolerance window ($\pm\%$). The highlighted marker points to the baseline design target ($\pm 3\%$ spec tolerance window yielding 54.8%). This demonstrates how tightly relaxing or tightening factory specifications directly impacts total product throughput.

5. Component Sensitivity to f_c Shift:



Uses Pearson correlation coefficients to measure and rank the individual impact of each component's tolerance on output f_c drift. The strongly negative correlation for C_1 and C_2 highlights capacitors as the primary drivers of variance, pointing out exactly where manufacturing control must focus.

6. Component vs. Output Spread (Box Plot):



Compares the normalized statistical dispersion (% deviation from nominal) of individual component values (R1,R2,C1,C2) against the resulting system output (fc). It clearly contrasts the tighter Gaussian distribution of resistors against the wider uniform spread of capacitors and shows how tolerance stacking propagates to fc.

Business ROI & Financial Impact

- **Bill of Materials (BOM) Cost Reduction**

Standard loose-tolerance capacitors (such as $\pm 6\%$ or $\pm 10\%$) cost a fraction of precision $\pm 1\%$ components. High-precision passive components carry up to a 5x price markup due to specialized manufacturing, tight sorting, and lower factory yields. By utilizing algorithmic pairing in software, the simulation proves that loose-tolerance components can meet strict $\pm 3\%$ circuit performance specs without paying a premium for tight-tolerance hardware.

- **Factory Yield Optimization**

Random assembly of wide-tolerance components causes a high percentage of completed circuit boards to drift out of target specs, forcing factory rejections. Smart binning ensures that high-deviation components are paired to counteract each other before soldering, driving total production yield close to 100% and maximizing factory throughput.

- **Scrap and Rework Minimization**

Failing end-of-line quality control tests incurs severe financial penalties through wasted raw materials, costly manual troubleshooting, and labor-intensive board rework. Minimizing out-of-spec filter assemblies drastically reduces parts-per-million (PPM) failure rates at the functional testing stage, preventing direct component scrap losses.

- **Zero-Hardware-Cost Innovation**

The entire yield improvement is accomplished via software intelligence without adding physical components, active trimming ICs, or manual potentiometer adjustments. The sorting algorithm operates virtually in code and transfers directly to automated SMT (Surface Mount Technology) pick-and-place machinery at zero extra hardware cost.

- **Capital Expenditures (CapEx) Avoidance**

Achieving tight circuit performance usually requires investing in expensive laser-trimming equipment or dedicated high-precision manufacturing lines. Smart combinatorial sorting leverages existing assembly line hardware, optimizing performance purely through intelligent software scheduling and component placement logic.

- **Supply Chain Flexibility**

Relying on rare $\pm 1\%$ high-precision components increases vulnerability to supplier shortages, lead-time delays, and price spikes. Enabling the design to pass strict specifications using commodity-grade parts allows sourcing from broader vendor bases, stabilizing supply chains and increasing purchasing leverage.

Discussion: In this project we have tried to implement all the ideas and theories we have learnt from this course. Moreover, during this project we have come across many issues to deal with. To solve such problems we have taken help from different sources. It helped us to make our project complete. This project helped us to understand the depth of many topics that we have learnt only in the theory. However, by using this simulation and idea, many companies can make profit from it.

End